



Connect Amazon Lex with Lambda

 Arpita Chowdhury

For which account would you like your balance?

amex

For verification purposes, what is your date of birth?

01/10/1999

Thank you. The balance on your Credit account is \$57.88 dollars.



Introducing Today's Project!

What is Amazon Lex?

Amazon Lex is a service from Amazon Web Services that lets you build conversational chatbots using voice or text, and it is useful because it handles natural language understanding and conversation flow automatically while integrating easily with backend services like AWS Lambda to perform real actions.

How I used Amazon Lex in this project

In today's project, I used Amazon Lex to build a conversational chatbot (BankerBot) that collects user input, understands intents, and integrates with AWS Lambda to dynamically return a bank account balance.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was how seamlessly Amazon Lex and AWS Lambda work together, and how easily a chatbot can be extended with real backend logic using code hooks and serverless functions without managing any infrastructure.



Arpita Chowdhury

NextWork Student

nextwork.org

This project took me...

This project took me 2 hours.



AWS Lambda Functions

AWS Lambda is a serverless service that runs backend code on demand without managing servers, commonly used with Amazon Lex to add real functionality to chatbots.

In this project, I created a Lambda function to generate a random bank balance and return it to my Amazon Lex chatbot when a user asks for their account balance. The function runs automatically when triggered by Lex, processes the user's request in the background, and sends the balance back so BankerBot can respond with a realistic, dynamic answer. This demonstrates how AWS Lambda adds backend logic to a chatbot using a serverless architecture, without managing any servers.



```
import json
import random
import decimal

def random_num():
    return(decimal.Decimal(random.randrange(1000, 50000))/100)

def get_slots(intent_request):
    return intent_request['sessionState']['intent']['slots']

def get_slot(intent_request, slotName):
    slots = get_slots(intent_request)
    if slots is not None and slotName in slots and slots[slotName] is not None:
        return slots[slotName]['value']['interpretedValue']
    else:
        return None

def get_session_attributes(intent_request):
    sessionState = intent_request['sessionState']
    if 'sessionAttributes' in sessionState:
        return sessionState['sessionAttributes']

    return {}
```



Chatbot Alias

An alias is a named pointer to a specific version of a Lambda function that allows services like Amazon Lex to invoke it safely and consistently.

TestBotAlias is an alias that points to the test version of the BankerBot, allowing Amazon Lex to safely interact with the bot during testing before it is deployed to production.

To connect Lambda with my BankerBot, I visited my bot's TestBotAlias and selected my Lambda function (BankingBotEnglish) under the Lambda function settings, then chose the \$LATEST version and saved the changes, because this allows Amazon Lex to invoke the Lambda function whenever the CheckBalance intent is triggered.

Alias language support: English (US)

▼ Lambda function - *optional*

This Lambda function is invoked for initialization, validation, and fulfillment.

Source

BankingBotEnglish ▼

Lambda function version or alias

\$LATEST ▼

[Learn more about Lambda](#) 

Cancel

Save

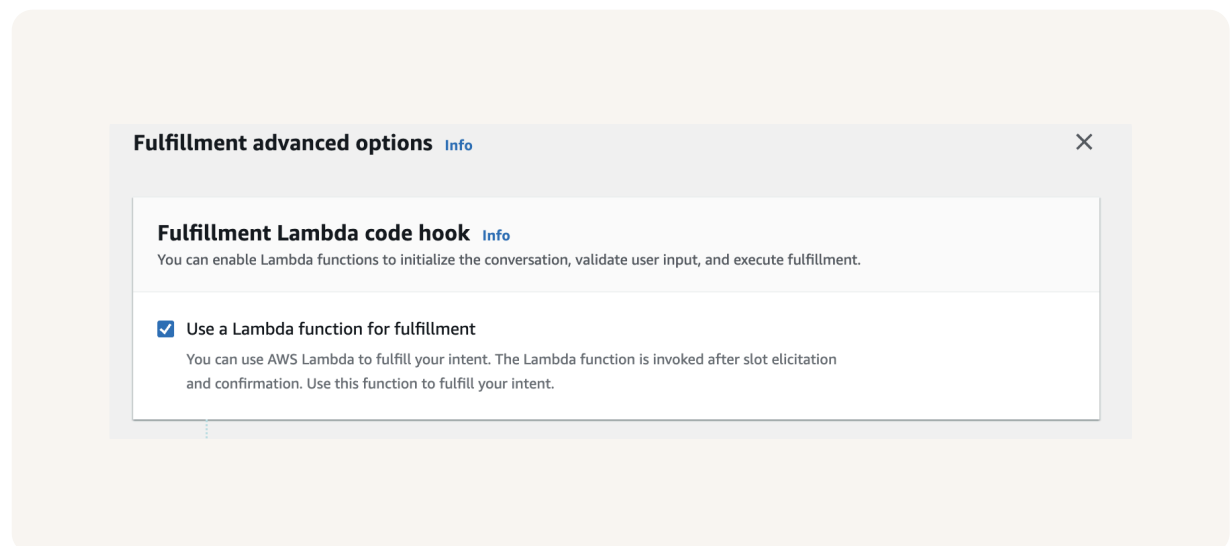


Code Hooks

A code hook is a way for Amazon Lex to invoke a Lambda function to handle logic that Lex cannot do on its own.

Even though I already connected my Lambda function with my chatbot's alias, I had to use code hooks because Lex still needs to be told exactly which intent should trigger the Lambda function and at what point in the conversation to run the backend logic.

I could find code hooks at the intent level inside Amazon Lex, specifically within the CheckBalance intent under the Fulfillment or Validation and Fulfillment settings.





The final result!

I've set up my chatbot to trigger Lambda and return a random dollar figure when a user triggers the CheckBalance intent by asking for their account balance (after providing their account type and birthday).

For which account would you like your balance?

amex

For verification purposes, what is your date of birth?

01/10/1999

Thank you. The balance on your Credit account is \$57.88 dollars.



Customizing the Lambda function

To level up the connection between Lambda and Lex, I updated my Python code to customize the chatbot's response by adding a fun, personalized message that randomly selects a description and dynamically inserts the user's account type and generated balance, making the interaction more engaging while still returning real-time data from Lambda.

Now, the message my users see when they ask for their bank balance is a personalized response that includes their account type, a randomly generated balance, and a fun descriptive phrase, making the chatbot feel more interactive and human-like. I can customise this message even further by adding conditional logic (for example, different messages for savings vs. checking accounts), formatting the balance differently, or pulling real data from a database instead of using a random number.



```
def CheckBalance(intent_request):
    session_attributes = get_session_attributes(intent_request)
    slots = get_slots(intent_request)
    account = get_slot(intent_request, 'accountType')
    #The account balance in this case is a random number
    #Here is where you could query a system to get this information
    balance = str(random_num())
    # Add a fun twist to the message
    descriptions = ["Arpita's top-secret digital vault", "your personal treasure box", "bank of NextWork"]
    description = random.choice(descriptions)
    text = f"Hello! Your {account} account balance, tucked away in {description}, is currently ${balance}."
    message = {
        'contentType': 'PlainText',
        'content': text
    }
    fulfillment_state = "Fulfilled"
    return close(intent_request, session_attributes, fulfillment_state, message)
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

